

Forward/Backward 메시지 방향 (p.15)

Forward-Backward Algorithm

```
# Step 1: Forward pass (left → right)
Compute F₁, F₂, …, Fₙ
  F₁(h₁) = p(h₁)·p(e₁|h₁)
  Fᵢ(hᵢ) = Σ_{h_{i-1}} F_{i-1}(h_{i-1})·p(hᵢ|h_{i-1})·p(eᵢ|hᵢ)
# Step 2: Backward pass (right → left)
Compute Bₙ, B_{n-1}, …, B₁
  Bₙ(hₙ) = 1
  Bᵢ(hᵢ) = Σ_{h_{i+1}} B_{i+1}(h_{i+1})·p(h_{i+1}|hᵢ)·p(e_{i+1}|h_{i+1})
# Step 3: Combine and normalize
for each i do
  Sᵢ(hᵢ) = Fᵢ(hᵢ)·Bᵢ(hᵢ)
  ℙ(Hᵢ=hᵢ | E=e) = Sᵢ(hᵢ) / Σ_v Sᵢ(v)      ← normalize
```

시간복잡도: O(n·|Domain|²) — DP로 공유 중간 계산. 모든 smoothing 질문을 한 번에 답변 가능.

★ 필수암기 Forward-Backward: F = start→h_i 경로합, B = h_i→end 경로합, S = F·B = h_i를 통과하는 전체 경로합. 정규화: $\mathbb{P}(H_i{=}h_i|E{=}e) = S_i(h_i) / \Sigma_v S_i(v)$. 복잡도 O(n·|Domain|²).

Particle Filtering p.20-28

Q. Propose·Weight·Resample 3단계?

Q. 왜 근사이며 언제 유리?

Q. 복잡도 O(nK)의 의미?

Forward-Backward의 한계 — 실수 위치값 등 |Domain| 크면 O(n·|Domain|²)이 너무 느림.

- Beam search: K개 최고 가중치 파티클 유지 — 탐욕적 prune이 *다양성* 파괴
- → Particle Filtering: propose→weight→resample 3단계로 해결

Particle Filtering 3단계 직관 — 파티클 집합으로 근사분포를 표현하고, 새 관측이 올 때마다 갱신.

Propose Weight Resample

- ① Propose: 전이분포 $p(h_i|h_{i-1})$으로 각 파티클에서 다음 상태 샘플링
- ② Weight: 방출분포 $p(e_i|h_i)$로 파티클에 가중치 부여
- ③ Resample: 가중치 비례로 K개 재추출 → 유망한 영역으로 재집중

Particle Filtering

```
Initialize  C ← [{}]    ← K개 빈 파티클
for i = 1, …, n do
  # Propose: 전이분포로 다음 상태 샘플링
  C' ← { h ∪ {Hᵢ: hᵢ} : h ∈ C, hᵢ ~ p(hᵢ | h_{i-1}) }
  # Weight: 관측 eᵢ에 의한 가중치 계산
  w(h) = p(eᵢ | hᵢ)  for each h ∈ C'
  # Resample: 가중치 비례로 K개 독립 추출
  C ← K particles drawn ∝ w(h) / Σ_{h'∈C'} w(h')
# 출력: normalize weights → 근사 ℙ(H₁,…,Hₙ | E=e)
```

filtering 질문: 마지막 H_i값만 유지해도 됨. 동일 파티클은 count로 압축 가능.

단계	핵심 아이디어	사용 분포
Propose	전이분포로 미래 상태 샘플 → 공간 탐색	$p(h_i \mid h_{i-1})$
Weight	관측 일치도로 파티클 점수 → 증거 반영	$p(e_i \mid h_i)$
Resample	가중치 비례 K개 재추출 → 다양성 유지	∝ weight

Particle Filtering 3단계 — Beam search(greedy prune)와 달리 확률적 재추출로 다양성 보존

Beam search vs Particle Filtering 비교

- Beam search: extend(모든 값 탐색=느림) + prune(greedy=다양성 손실)
- Particle Filtering: propose(전이분포 샘플=효율적) + resample(확률적=다양성 유지)
- Particle Filtering: |Domain| 무관 — 대규모 연속 공간으로 확장 가능

⚠ **함정** Particle Filtering은 근사 알고리즘 — K(파티클 수)가 작으면 분산이 커진다. Forward-Backward와 달리 정확한 답을 보장하지 않음.

★ **필수암기** Forward-Backward: 정확·O(n|Domain|²) → 소규모 이산 도메인. Particle Filtering: 근사·O(nK) → 대규모·연속 도메인. 3단계 키워드: Propose(전이분포 샘플)·Weight(방출분포 가중)·Resample(비례 재추출).

요약 BN 추론 5단계(evidence 조건화→leaf 제거→단절 제거→Markov net 변환→알고리즘) · HMM: **Forward-Backward**(lattice DP, O(n|D|²), 정확) vs **Particle Filtering**(Propose→Weight→Resample, O(nK), 근사·대규모).

Ch 16 Bayesian Networks 3 — 파라미터 학습

Supervised Learning · Maximum Likelihood · Laplace Smoothing · EM Algorithm

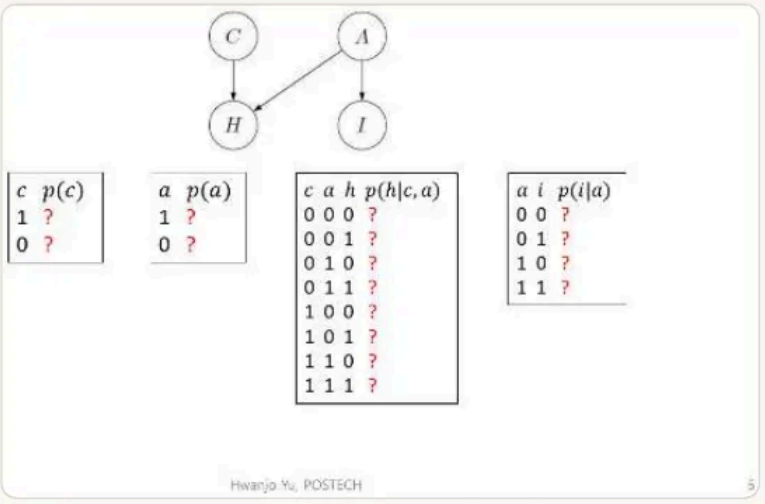
학습 과제 (Learning Task) p.5-6

Q. parameter learning이 묻는 것은?

Q. 완전관측 vs 부분관측 데이터?

파라미터 학습이란 — BN 구조(DAG)는 주어진 상태에서, 각 노드의 local conditional distribution θ를 훈련 데이터로부터 추정하는 문제.

- **입력**: 완전 배정(full assignments)의 훈련 데이터 `D`
- **출력**: 파라미터 집합 `θ = {p_d : d ∈ D}` (e.g., Genre, Rating)
- 각 변수 `x_i` 는 분포 `p_{d_i}` 에서 생성 — parameter sharing 가능(여러 변수가 같은 θ 사용)



BN 구조와 CPT 예시 — 각 노드마다 물음표 파라미터를 추정 (p.5)

✎ **핵심** 파라미터 수: 루트 노드는 값 수 −1, 내부 노드는 (부모 조합 수) × (값 수 −1). Naïve Bayes — `2 + 2·|V|` params; HMM — `|S| + |S|² + |S|·|O|` params.

Maximum Likelihood — Count & Normalize p.7-22

Q. MLE의 count & normalize 절차?

Q. closed-form인 이유는?

직관 — 비율로 추정 — 각 조건부 확률은 해당 조합이 데이터에 등장한 횟수의 비율.

- 단일 변수: `p(x=v) ∝ count(v)` — 전체 N개 중 v 등장 횟수
- 조건부: `p(x_i | x_Parents(i)) ∝ count(x_Parents(i), x_i)`
- 각 분포를 *독립적으로* 추정 가능 → 문제가 decompose됨

Count & Normalize (Maximum Likelihood)

```
Input:  D  (full assignments), θ = {p_d : d ∈ D}
Output: θ (ML parameters)
# Count phase
for each x ∈ D:
  for each variable x_i:
    Increment count_d_i(x_Parents(i), x_i)
# Normalize phase
for each d and local assignment x_Parents(i):
  Set p_d(x_i | x_Parents(i)) ∝ count_d(x_Parents(i), x_i)
```

각 분포 d를 독립 subproblem으로 풀 수 있다. 로그를 취해 미분하면 closed-form 해가 나옴.

Maximum Likelihood 목적함수 — 모든 데이터의 결합 확률을 최대화하는 θ 탐색.

- 목적: `max_θ ∏_{x∈D} P(X=x ; θ)`
- 분해: `max p_G · max p_R|G · ...` — 각 분포마다 독립 최적화
- 해: closed-form — count and normalize가 정확히 MLE

⚠ **함정** MLE 문제점 — 데이터에 한 번도 등장하지 않은 값/조합에 대해 확률 0 배정 → *Overfitting*. 예: 훈련에 (comedy, 2) 없으면 p_R(2|c)=0, 실제론 가능한 값인데 완전히 배제됨.

Laplace Smoothing p.24-28

Q. 0-확률 문제와 +λ의 효과?

Q. λ가 추정에 미치는 영향은?

핵심 아이디어 — 가상 카운트 추가 — 각 조합마다 λ개의 가상(hallucinated) 데이터를 추가해 0-확률을 방지.

λ=1: add-one
λ→∞: uniform

- 수식: `count_d(x_Parents, x_i) ← count_d(...) + λ`
- λ=1이 가장 흔함 ("add-one smoothing")
- 분모도 λ × (값 개수)만큼 증가 → 정규화 유지

λ 크기와 데이터의 상호작용 — λ가 클수록 균등분포에 가까워지고, 데이터가 많을수록 smoothing이 희석됨.

- 소량 데이터: λ가 확률을 dominant → 근사 균등분포
- 대량 데이터: data wins out → MLE에 수렴 (1+998 ≈ 0.999)

케이스	count(d)	count(c)	p(d)	p(c)
λ=1/2, n=1개	1/2+1	1/2	3/4	1/4
λ=1, n=1개	1+1	1+1	2/3 (?)	1/3 (?)
λ=1, n=999개	1+998	1	≈0.999	≈0.001

λ=1, 소량 데이터 vs 대량 데이터 — 데이터가 많으면 smoothing이 흡수됨 (p.27)

★ **필수암기** Laplace smoothing 공식: `p_d(x_i | x_Par) = (count(...) + λ) / (Σ count(...) + λ·|dom(x_i)|)` . λ=0이면 MLE. 모든 unseen 조합에 >0 확률 보장.

EM Algorithm — 미관측 변수 p.30-38

Q. E-step·M-step 각각 무엇?

Q. 왜 반복? 무엇이 단조증가?

Q. local optimum 수렴의 의미?

동기 — Hidden Variable — 일부 변수가 관측되지 않을 때 MLE(*count & normalize*)를 직접 적용 불가.

- 완전 관측: `D = {(d,4,5),(d,4,4),...}` → 바로 MLE
- Hidden G: `D = {(?,4,5),(?,4,4),...}` → Genre를 모름
- 목표: 최대 주변 우도(maximum marginal likelihood) = `max_θ Σ_h P(H=h, E=e ; θ)`

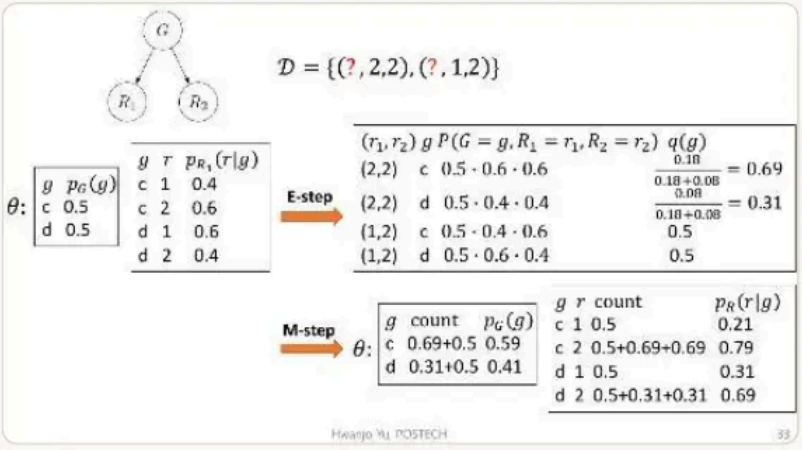
Expectation Maximization (EM)

```
Input:  observed data D = {e^(1), ..., e^(n)},  BN structure
Output: parameters θ
Initialize θ randomly
repeat until convergence:
  # E-step: soft-완성 (probabilistic inference)
  for each example e in D:
    for each hidden value h:
      q(h) = P(H=h | E=e ; θ)      ← 현재 θ로 추론
    Create weighted example (h, e) with weight q(h)
  # M-step: weighted MLE
  θ = count_and_normalize(weighted examples)      ← 분수 카운트
```

수렴 보장: 각 반복에서 marginal likelihood가 단조 증가. K-means는 EM의 특수 케이스 (hard E-step).

E-step vs M-step 역할

- E-step: 현재 θ로 hidden `H` 의 사후 분포 `q(h)` 계산 → 소프트 완성(soft completion) — 각 예제가 여러 hidden 값에 분산
- M-step: 가중 카운트(`fractional count`) → count & normalize → 새 θ
- 반복: θ → q(h) → θ → q(h) ... 수렴 (local optimum 보장)



EM 1 iteration — D={(?,2,2),(?,1,2)}, E-step 가중 배정 → M-step count & normalize (p.33)

⚠ **함정** EM은 local optimum에 수렴 — global 최적 보장 없음. 초기값 θ에 따라 결과 달라질 수 있음 → 여러 random init 권장.

응용 — 암호 해독 (Decipherment as HMM) p.34-38

Q. decipherment를 HMM으로 보는 법?

Q. EM이 여기서 하는 역할은?

치환 암호 → HMM으로 모델링 — 평문 문자열(`H_1,...,H_n`)이 hidden, 암호문(`E_1,...,E_n`)이 observed.

- 파라미터 `θ = (p_init, p_trans, p_emit)`
- p_init: 균등분포로 고정
- p_trans: 대량 영어 텍스트로 추정 (언어 모델)
- p_emit: 치환 테이블 — EM으로 추정

EM 적용

- E-step: forward-backward로 각 위치 `t` 와 문자 `h` 에 대해 `q_t(h) = P(H_t=h | E=e ; θ)` 계산
- M-step: `count_emit(h, c) = Σ_t q_t(h)·[E_t=c]` → `p_emit(c|h) ∝ count_emit(h, c)`
- 직관: *언어 모델이 영어다운 평문 h를 선호 + p_emit이 일관된 치환을 선호 → 수렴*

학습 알고리즘 비교 p.20-32

Q. MLE vs Laplace vs EM 적용 조건?

	MLE (Count & Normalize)	MLE + Laplace	EM
관측 조건	완전 관측	완전 관측	일부 hidden 가능
핵심 연산	count → normalize	count+λ → normalize	E-step + M-step 반복
0-확률	가능 (overfitting)	불가 (λ>0)	가능
수렴	Closed-form (1회)	Closed-form (1회)	반복, local optimum
복잡도	O(|D|·n)	O(|D|·n)	O(iter · inference)

세 학습 알고리즘 비교. EM은 hidden 변수 처리를 위해 반복 추론 필요.

★ 필수암기 MLE = count & normalize, closed-form, 완전관측 필수. Laplace = +λ hallucinated, 0-prob 방지, λ 크면 uniform 편향. EM = E-step(soft 추론)+M-step(weighted MLE) 반복, hidden variable 처리, local opt 수렴. K-means ⊂ EM (hard E-step).

요약 BN 파라미터 학습 · **MLE=count & normalize**(closed-form, 완전관측) · Laplace smoothing(+λ로 0-확률 방지) · **EM = E-step(soft 추론)+M-step(weighted MLE) 반복**으로 hidden var 처리 → local optimum 수렴.

Ch 17 Logic 1 — Propositional Logic

Syntax · Semantics · Knowledge Base · Entailment · Satisfiability

Logic-Based Models p.3-6

Q. logic 기반 모델의 목적은?

Q. syntax vs semantics 구분?

3가지 모델링 패러다임 — AI 모델의 스펙트럼 — 표현력 ↔ 계산 효율 트레이드오프.

- State-based — search, MDP, game: state·action·cost로 사고
- Variable-based — CSP, Bayes net: variable·factor로 사고
- Logic-based — propositional / first-order logic: formula + inference rules로 사고 · *High-level intelligence*

Logic의 두 목표 — ① 세계에 대한 지식을 표현 (Represent knowledge) · ② 그 지식으로 추론 (Reason with knowledge).

✎ 핵심 Logic의 핵심 트레이드오프: 표현력(expressivity) ↑ → 계산 효율 ↓. Horn clause 제한 propositional → 완전 propositional → first-order → second-order 순으로 표현력 증가.

Syntax of Propositional Logic p.16-18

Q. atomic·connective 구성요소는?

Q. formula 재귀적 정의는?

Syntax 정의 — 유효한 formula(수식)의 집합을 정의. 아직 의미(semantics) 없음 — 순수 기호.

- Propositional symbols (atomic): A, B, C, … — 더 이상 나눌 수 없는 명제 단위
- Logical connectives: ¬ (negation), ∧ (conjunction), ∨ (disjunction), → (implication), ↔ (biconditional)

Formula 재귀 구성 규칙 — f, g가 formula이면 ¬f, f∧g, f∨g, f→g, f↔g 도 formula (recursive definition).

- Formula 예: ¬A, ¬A→B, ¬A∧(¬B→C)∨(¬C∨D), ¬¬¬A
- Non-formula: ¬ (피연산자 없음), A+B ('+' 는 propositional connective 아님)

Semantics — Model & Interpretation p.19-25

Q. interpretation/model w란?

Q. I(f,w) 재귀 평가 방식은?

Q. f가 만족된다는 의미는?

Model w — 명제 기호에 진릿값을 대입한 완전 할당(assignment).

model = 세계의 한 상태

- n개 기호 → 2^n개 가능한 model
- 예: {A:1, B:1, C:0} — 하나의 model

Interpretation function I(f, w) — formula f와 model w를 받아 **true/false** 반환. 재귀 정의로 복합 formula를 원자부터 평가.

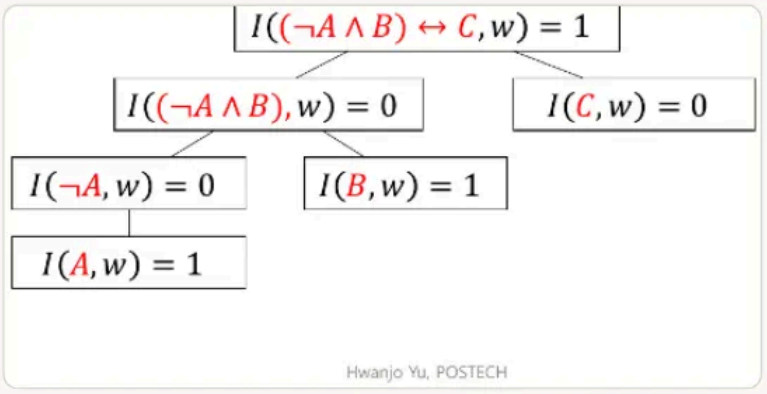
- Base: I(p, w) = w(p) ← 원자 기호는 model의 값을 그대로
- Recursive: 아래 진리표로 ¬f, f∧g, f∨g, f→g, f↔g 계산

I(f,w)	I(g,w)	I(¬f,w)	I(f∧g,w)	I(f∨g,w)	I(f→g,w)	I(f↔g,w)
0	0	1	0	0	1	1
0	1	1	0	1	1	0
1	0	0	0	1	0	0
1	1	0	1	1	1	1

진리표 — implication f→g: f=0이면 무조건 1 (vacuously true) (p.22)

Parse tree로 재귀 평가 (Worked Example) — f = (¬A∧B)↔C, w = {A:1, B:1, C:0}

- I(A,w)=1 → I(¬A,w)=0 → I(¬A∧B,w)=0∧1=0 → I(f,w)=I(0↔0,w)=1



Parse tree 재귀 평가 예시 — bottom-up으로 진릿값 전파 (p.23)

M(f) — formula가 나타내는 model 집합 — M(f) = {w | I(f,w)=1}. formula는 **model 집합을 컴팩트하게 표현**하는 수단.

- 예: f = (A∧B)↔C → M(f) = {{A:0,B:0,C:0}, {A:1,B:0,C:0}, {A:0,B:1,C:0}, {A:1,B:1,C:1}}

Knowledge Base (KB) p.26-28

Q. KB를 model 집합으로 보는 관점?

Q. KB가 model 교집합인 이유?

Knowledge Base 정의 — formula들의 집합 — 교집합(conjunction)으로 해석: M(KB) = ∩_{f∈KB} M(f). KB는 **세계에 대한 제약 조건** 집합.

- 예: KB = {Rain∨Snow, Traffic} → M(KB) = M(Rain∨Snow) ∩ M(Traffic)
- formula 추가 → M(KB) **축소** (더 강한 제약) — "더 많이 알수록 후보 세계가 줄어든다"

Entailment · Contradiction · Contingency p.29-32

Q. entailment KB⊨f 정의는?

Q. contradiction·contingency 구분?

Q. Venn 다이어그램 해석은?

Entailment — KB ⊨ f — KB가 f를 **함의**: M(KB) ⊆ M(f). f가 아무 정보도 추가 안 함 (이미 알려진 사실).

- 예: Rain∧Snow ⊨ Snow

Contradiction — KB가 f와 **모순**: M(KB) ∩ M(f) = ∅. f는 KB와 양립 불가.

- 예: Rain∧Snow contradicts ¬Snow
- **명제**: KB contradicts f ⟺ KB ⊨ ¬f

Contingency — f가 KB에 **비자명 정보를 추가**: ∅ ⊂ M(KB)∩M(f) ⊂ M(KB).

- 예: KB에 Rain이 있을 때 Snow는 contingent